Lab 10 - Train a support classifier

AI Agents for Business | TGS-2023018987 | v1.0

Objective

Train and compare real Hugging Face classifiers and a transparent baseline.

Alignment: K5, K9, K13; A4, A5, A6. Scheduled hands-on time: 50 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- train.csv
- validation.csv
- test.csv
- vocab.txt
- requirements.txt

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Create and activate a local virtual environment, then install requirements.txt. This exercise uses a small randomly initialised BERT classifier and synthetic labelled support messages; it is not a production pretrained model.

2. Inspect the three CSV splits. Check that record IDs and text strings do not overlap. Keep the test split untouched while choosing settings.

3. Run python run.py. The script trains two transformer configurations on CPU and a count-vector baseline; validation macro-F1 selects the transformer configuration.

4. Inspect training-log.json for loss, learning rate, dropout, clipping and validation metrics. Explain what backpropagation changes and why validation controls selection.

5. Open output.json and compare final held-out metrics, confusion matrices and the model limitations. A poor score is valid evidence, not a reason to edit the test labels.

6. Inspect the saved selected-model directory and tokenizer. Change one learning-rate or dropout setting, run an additional experiment and compare validation evidence; keep the final test for the final report.

7. Use PROMPTS.md to critique your strategy and document why Hugging Face helps with reproducible tokenization, model configuration and save/load behaviour. Submit code plus evidence as the practical build record.

Acceptance checks

- Training runs use real gradients and save a reloadable model.
- Training/validation/test IDs and texts are disjoint.
- Two strategies and a count baseline are compared; selection uses validation only.

Run python verify.py after run.py (use python3 if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's requirements.txt.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.
- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after assessment; keep source data and prompts unchanged.